

W2 PRACTICE

Native HTTP and Manual Routing

 *At the end of this practice, you can*

- ✓ **Create** and run a native Node.js HTTP server
- ✓ **Manually implement** route handling using conditionals.
- ✓ Serve **static files** using fs.
- ✓ Parse form data from POST requests.
- ✓ Debug and enhance server code using console outputs.

 *Get ready before this practice!*

- ✓ **Read** the following documents to understand Nodejs built-in HTTP module:
<https://nodejs.org/api/http.html>
- ✓ **Read** the following documents to understand Anatomy of an HTTP Transaction:
<https://nodejs.org/en/learn/modules/anatomy-of-an-http-transaction>

 *How to submit this practice?*

- ✓ Once finished, push your **code to GITHUB**
- ✓ Join the **URL of your GITHUB** repository on LMS



EXERCISE 1 – ANALYZE

Goal

- ✓ Identify and fix the bug.
- ✓ Understand the request-response cycle in Node.js using the `http` module.
- ✓ Explain the role of `res.write()` and `res.end()` in sending data back to the client.



For this exercise, you are provided with a minimal `server.js` file. Read and run the code and observe how it behaves.

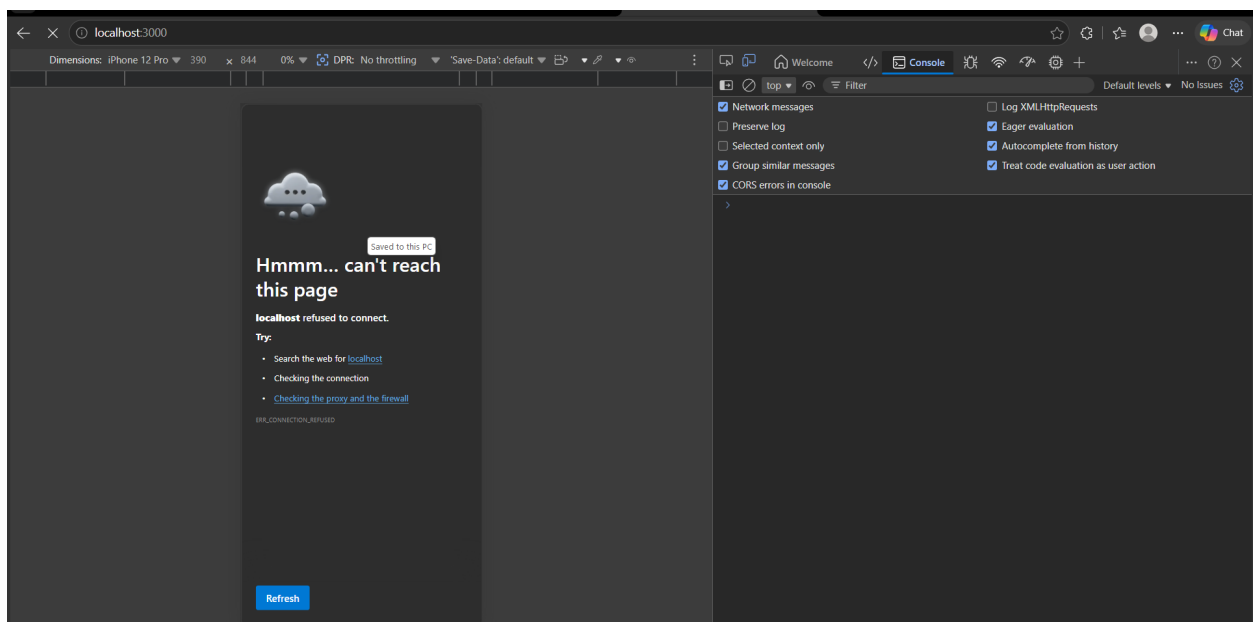
```
// server.js
const http = require('http');

const server = http.createServer((req, res) => {
  res.write('Hello, World!');
  return res.endd();
});

server.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

Q1 – What error message do you see in the terminal when you access `http://localhost:3000`? What line of code causes it?

The `return res.endd();` is incorrect.
This line of code causes it. `return res.endd();`



Q2 – What is the purpose of `res.write()` and how is it different from `res.end()`?

`res.write()` is for sending back the response to the client
`res.end()` is for finish the response and close the response or connection

Q3 – What do you think will happen if `res.end()` is not called at all?

I think the server will not send any response to the client even though the `res.write()` is already called since the server still think there would be more responses and not end yet

Q4 – Why do we use `http.createServer()` instead of just calling a function directly?

Because we need to add the http

Q5 – How can the server be made more resilient to such errors during development?

To make the server more resilient to such errors during development, we should use some ways such as:

- Use try-catch block to prevent the server from crashing when there is unexpected errors happen
- Use nodemon npm to automatically restart the server when files change, so there would be easier to debug.

EXERCISE 2 – MANIPULATE

Goal

- ✓ Practice using `req.url` and `req.method`.
- ✓ Understand how manual routing mimics what frameworks (like Express) automate.
- ✓ Serve both plain text and raw HTML manually.

🔧 For this exercise you will start with a **START CODE (EX-2)**

TASK 1 - Update the code above to add custom responses for these routes:

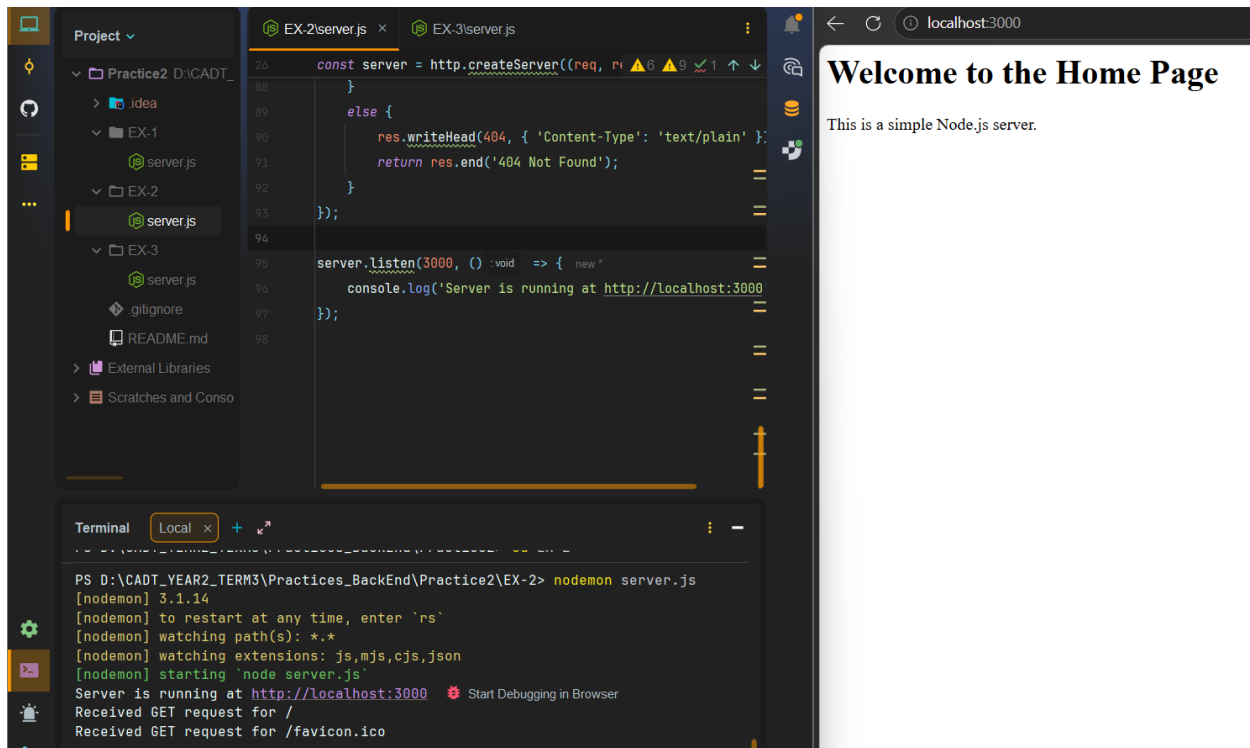
ROUTE	HTTP METHOD	RESPONSE
/about	GET	About us: at CADT, we love node.js!
/contact-us	GET	You can reach us vai email...
/products	GET	Buy one get one...
/projects	GET	Here are our awesome projects

Use [VS Code's Thunder Client](#) (or other tools (POSTMAN, INSOMIA) of your choice or curl on your terminal to make request.

Example output

```
curl http://localhost:3000/about ----->
About us: at CADT, we love node.js!
```

```
curl http://localhost:3000/contact-us -----
-> You can reach us vai email...
```



Project

Practice2 D:\CADT_> .idea> EX-1> server.js> EX-2> server.js> EX-3> server.js> .gitignore> README.md> External Libraries> Scratches and Conso

EX-2server.js

EX-3server.js

```
26 const server = http.createServer((req, res) => {
37   // Implement more routes here
38   if (url === '/about' && method === 'GET') {
39     res.writeHead(200, { 'Content-Type': 'text/html' });
40     return res.end(`
41       <html>
42         <head><title>About Us</title></head>
43         <body>
44           <h1>at CADT, we love node.js!</h1>
45         </body>
46       </html>
47     `);
48   }
49 });
50 // Implement more routes here
51 if (url === '/about' && method === 'GET') {
52   res.writeHead(200, { 'Content-Type': 'text/html' });
53   return res.end(`
54     <html>
55       <head><title>About Us</title></head>
56       <body>
57         <h1>at CADT, we love node.js!</h1>
58       </body>
59     </html>
60   `);
61 }
62 // Implement more routes here
63 if (url === '/products' && method === 'GET') {
64   res.writeHead(200, { 'Content-Type': 'text/html' });
65   return res.end(`
66     <html>
67       <head><title>Products</title></head>
68       <body>
69         <h1>Buy one get one...</h1>
70       </body>
71     </html>
72   `);
73 }
74 // Implement more routes here
```

Terminal

Local

PS D:\CADT_YEAR2_TERM3\Practices_BackEnd\Practice2\EX-2> node server.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Server is running at http://localhost:3000
Received GET request for /
Received GET request for /favicon.ico
Received GET request for /about

localhost:3000/about

at CADT, we love node.js!

Project

Practice2 D:\CADT_> .idea> EX-1> server.js> EX-2> server.js> EX-3> server.js> .gitignore> README.md> External Libraries> Scratches and Conso

EX-2server.js

EX-3server.js

```
26 const server = http.createServer((req, res) => {
37   // Implement more routes here
38   if (url === '/about' && method === 'GET') {
39     res.writeHead(200, { 'Content-Type': 'text/html' });
40     return res.end(`
41       <html>
42         <head><title>About Us</title></head>
43         <body>
44           <h1>at CADT, we love node.js!</h1>
45         </body>
46       </html>
47     `);
48   }
49 });
50 // Implement more routes here
51 if (url === '/about' && method === 'GET') {
52   res.writeHead(200, { 'Content-Type': 'text/html' });
53   return res.end(`
54     <html>
55       <head><title>About Us</title></head>
56       <body>
57         <h1>at CADT, we love node.js!</h1>
58       </body>
59     </html>
60   `);
61 }
62 // Implement more routes here
63 if (url === '/products' && method === 'GET') {
64   res.writeHead(200, { 'Content-Type': 'text/html' });
65   return res.end(`
66     <html>
67       <head><title>Products</title></head>
68       <body>
69         <h1>Buy one get one...</h1>
70       </body>
71     </html>
72   `);
73 }
74 // Implement more routes here
```

Terminal

Local

Server is running at http://localhost:3000
Received GET request for /
Received GET request for /favicon.ico
Received GET request for /about
Received GET request for /contact-us
[nodemon] restarting due to changes...
Received GET request for /contact-us
[nodemon] starting `node server.js`
Server is running at http://localhost:3000
Received GET request for /contact-us

localhost:3000/contact-us

You can reach us vai email...

Project

Practice2 D:\CADD\

idea

EX-1

server.js

EX-2

server.js

EX-3

server.js

gitignore

README.md

External Libraries

Scratches and Conso

EX-2server.js

EX-3server.js

26

const server = http.createServer((req, res) => {

58

return res.end(`

59

<html>

60

<head><title>Contact Us</title></head>

61

<body>

62

<h1>You can reach us vai email...</h1>

63

</body>

64

</html>

65

`

66

)

67

url === '/products' && method === 'GET') {

68

res.writeHead(200, { 'Content-Type': 'text/html' });

69

return res.end(`

70

<html>

71

<head><title>Products</title></head>

72

<body>

73

<h1>Buy one get one...</h1>

74

</body>

75

</html>

76

`

77

)

78

url === '/projects' && method === 'GET') {

79

res.writeHead(200, { 'Content-Type': 'text/html' });

80

return res.end(`

81

<html>

82

<head><title>Projects</title></head>

83

<body>

84

<h1>Here are our awesome projects</h1>

85

</body>

86

</html>

87

`

88

)

89

})

Terminal

Local x

Received GET request for /

Received GET request for /favicon.ico

Received GET request for /about

Received GET request for /contact-us

[nodemon] restarting due to changes...

Received GET request for /contact-us

[nodemon] starting 'node server.js'

Server is running at http://localhost:3000 Start Debugging in Browser

Received GET request for /contact-us

Received GET request for /products

localhost:3000/products

Buy one get one...

EX-2server.js

EX-3server.js

26

const server = http.createServer((req, res) => {

73

<h1>Buy one get one...</h1>

74

</body>

75

</html>

76

`

77

)

78

url === '/projects' && method === 'GET') {

79

res.writeHead(200, { 'Content-Type': 'text/html' });

80

return res.end(`

81

<html>

82

<head><title>Projects</title></head>

83

<body>

84

<h1>Here are our awesome projects</h1>

85

</body>

86

</html>

87

`

88

)

89

})

Terminal

Local x

Received GET request for /favicon.ico

Received GET request for /about

Received GET request for /contact-us

[nodemon] restarting due to changes...

Received GET request for /contact-us

[nodemon] starting 'node server.js'

Server is running at http://localhost:3000 Start Debugging in Browser

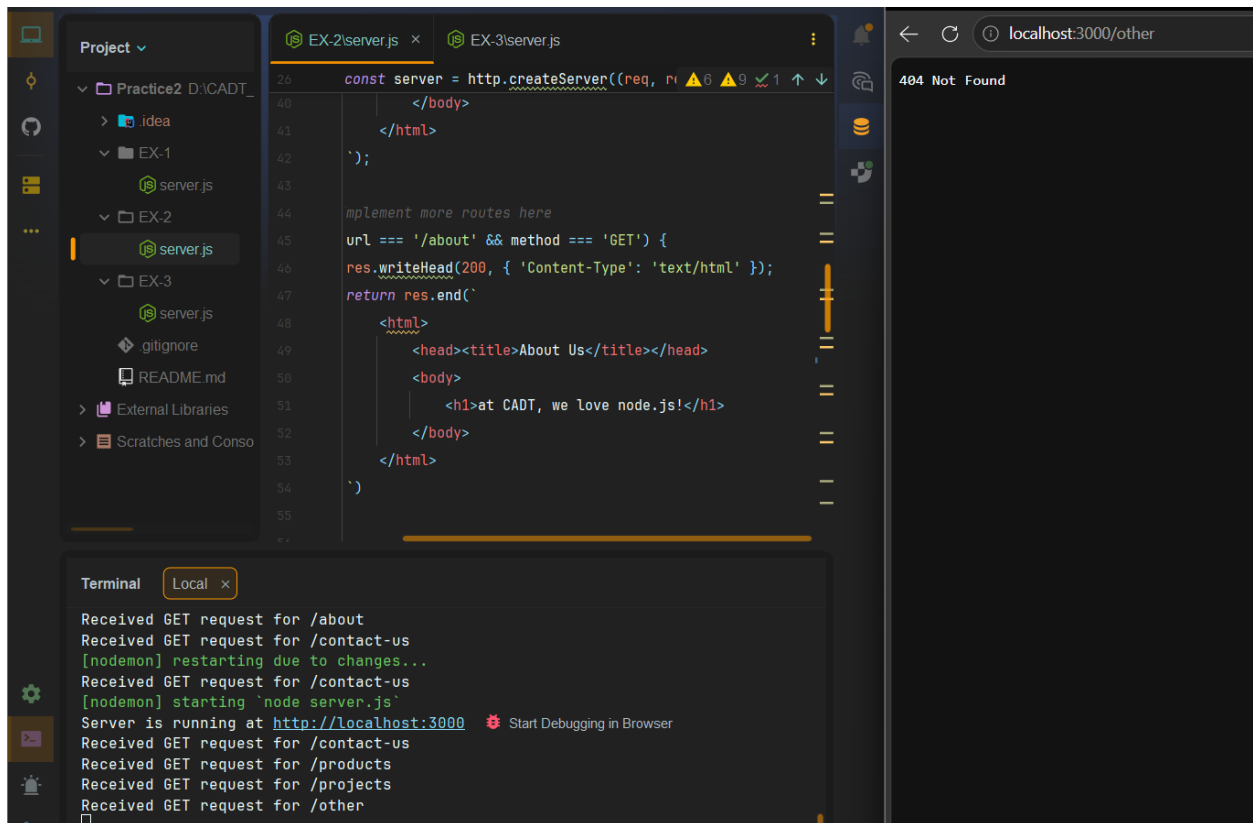
Received GET request for /contact-us

Received GET request for /products

Received GET request for /projects

localhost:3000/projects

Here are our awesome projects



TASK 2 – As we can see the complexity grow as we add more routes. Use `switch` statement to arrange the code into more organized structure.

? Reflective Questions

1. What happens when you visit a URL that doesn't match any of the three defined?

The page will show '404 Page Not Found'

2. Why do we check both the `req.url` and `req.method`?

Because the same URL can behave differently depending on the HTTP method.

3. What MIME type (`Content-Type`) do you set when returning HTML instead of plain text?

```
res.setHeader('Content-Type', 'text/html');
```

4. How might this routing logic become harder to manage as routes grow?

As more routes are added there will be many if-else or switch cases appear, code becomes repetitive, debugging becomes harder, organizing routes gets messy.

5. What benefits might a framework offer to simplify this logic?

Frameworks help by providing:

- easier routing syntax,
- middleware support,
- better code organization,
- automatic error handling,
- cleaner request/response handling,
- support for templates, APIs, and databases.

EXERCISE 3 – CREATE

Goal

- ✓ Practice handling `POST` requests.
- ✓ Parse URL-encoded form data manually.
- ✓ Write and append to local files using Node.js' `fs` module.
- ✓ Handle async operations and errors gracefully.

🔗 For this exercise you will start with a **START CODE EX-3**

TASK 1 - Extend your Node.js HTTP server to handle a **POST request** submitted from the contact form. When a user submits their name, the server should:

1. **Capture the form data** (from the request body).
2. **Log it to the console**.
3. **Write it to a local file** named `submissions.txt`.

Testing, go to `/contact` on browser and test

Requirements

- Handle `POST /contact` requests.
- Parse raw `application/x-www-form-urlencoded` data from the request body.
- Write the name to a new line in `submissions.txt`.
- Send a success response to the client (HTML or plain text).

4. Why do we use `fs.appendFile` instead of `fs.writeFile`?

Because `fs.appendFile` added data, but `fs.writeFile` is replacing data. In this case, using `fs.appendFile` is the better option.

5. How could this be improved or made more secure?

- Validate input
- Limit input size
- Sanitize data (avoid injection)
- Use JSON instead of raw text
- Use database instead of file
- Handle Errors Better
- Use HTTPS

Bonus Challenge (Optional)

- Validate that the `name` field is not empty before saving.
- Send back a small confirmation HTML page instead of plain text.
- Try saving submissions in JSON format instead of plain text.